

Approach to Enhancing Concurrent and Self-Reliant Access to Cloud Database: A Review

Vivek Waghmare, Kaveri Gojre, Akshaya Watpade

Vivek.waghmare@sitrc.org

kaverigojre@gmail.com

akshu8.watpade@gmail.com

Sandip Institute of Technology and Research Centre, Nasik-422213, Maharashtra, India

Abstract—Now a day's cloud computing is power station to run multiple businesses. It is cumulating more and more users every day. Database-as-a-service is service model provided by cloud computing to store, manage and process data on a cloud platform. Database-as-a-service has key characteristics such as availability, scalability, elasticity. A customer does not have to worry about database installation and management. As a replacement, the cloud database service provider takes responsibility for installing and maintaining the database. The real problem occurs when it comes to storing confidential or private information in the cloud database; we cannot rely on the cloud data vendor. A curious cloud database vendor may capture and leak the secret information. For that purpose, Protected Database-as-a-service is a novel solution to this problem that provides provable and pragmatic privacy in the face of a compromised cloud database service provider. Protected Database-as-a-service defines various encryption schemes to choose encryption algorithm and encryption key to encrypt and decrypt data. It also provides "Master key" to users, so that a metadata storage table can be decrypted only by using the master key of the users. As a result, a cloud service vendor never gets access to decrypted data, and even if all servers are jeopardized, in such inauspicious circumstances a cloud service vendor will not be able to decrypt the data.

Proposed Protected Database-as-a-service system allows multiple geographically distributed clients to execute concurrent and independent operation on encrypted data and also conserve data confidentiality and consistency at cloud level; to eradicate any intermediate server between the client and the cloud database.

Keywords— Advance Encryption Standard, Cloud Computing, confidentiality, Database-as-a-Service, Master key, Protected Database-as-a-Service, Security.

I. INTRODUCTION

Cloud database is a database which is stored on remote server rather than local and accessed using internet. Cloud database presents three prevalent deployment models:

- Virtual Machine Image
- Database-as-a-Service
- Managed DB Hosting on Cloud

In virtual machine image storage space is provided by cloud service provider where user has to upload its machine image where it is possible to run database or user can buy ready machine image which has database already installed in it. The second deployment Database-as-a-service, here it is not necessary to user to launch machine image and install database, cloud database vendor install and maintain database for service user. In managed database hosting cloud provider handles database on the behalf of user. In this environment imperative information is placed in infrastructure of untrusted service provider, restricting data is predominant importance. This requirement urges intelligible management choices: data must be accessed only by authenticated users that do not include cloud database vendor and intermediaries. In any intermediary context data must be in encrypted. There are various to achieve data secrecy in Database-as-a-Service paradigm. But security or privacy in database-as-a-service [1] is still open research.

In this context we proposed Protected Database-as-a-Service system which allows user to take complete benefit of cloud services and still achieve data confidentiality without releasing data to cloud database service provider. In Protected Database-as-a-Service geographically distributed users can concurrently and independently access the cloud database. Protected Database-as-a-Service conserve rules that promise information access and consistency at cloud level. Many existing systems are based on the principal of middle level proxy or service brokers. Protected Database-as-a-Service abolishes single middle level trusted proxy servers or service brokers to achieve higher reliability without bargaining with basic

features provided by Database-as-a-Service such as accessibility, scaling and availability. Single application server is vulnerable to the system catastrophe. But Protected Database-as-a-Service unremitting work hiding any failure from client. This is practicable by cloning application server through which client would be converse with cloud database. Protected Database-as-a-Service architecture has the potential of executing SQL statement on encrypted data. Alteration of encrypted database structure is supported by Protected Database-as-a-Service. Protected Database-as-a-Service provides various qualities that distinguish it from existing work.

- It is possible to perform concurrent SQL operation on encrypted cloud database.
- It provides all features same as Database-as-a-Service because it abolishes failure of middle level server.
- Data can be access separately and at the same time from remote location by multiple clients.
- Cloning of middle level application server so rapid deployment of client request.
- Disasters does not influence client requests.
- Compatible with all popular relational database servers.

II. RELATED WORK

Following part of paper represents detailed description of the earliest work in this system.

A. SPORC, SUNDR, DEP

A.J. Feldman, W.P. Zeller, M.J. Freedman, and E.W. Felten proposed the SPORC system. SPORC is an inclusive substructure for construction of an extensive diversity of cooperative approaches with unfaithful services vendors. A server perceives only encrypted data and cannot drift from veracious implementation without being recognized. It sanctions simultaneous, tolerates detached behaviour, braces constant dynamic access control yet in the existence of concurrency and knee-high latency improving of split condition. SPORC can be revealed pliability through two formation supplication: First one is a causally-consistent key-value store and the second one is a browser-based collaborative text editor [2]. DEP (Cloud Storage with Minimal Trust) Depot permits malevolent etiquette by any number of users or cloud database vendor. Besides, it furnishes welfare and aliveness assurances to correct users. Depot dispenses these assurances using two-layer architecture. In first layer Depot clinches that the updates perceived by unerring nodes are unchanging ordered under Fork-Join-Causal consistency (FJC).

SUNDR (Secure Untrusted Data Repository) is a secure network file system. SUNDR promises the uprightness of data even when malevolent parties dominance the cloud server. SUNDR breaks storage task between two unfaithful elements, first one is a block store and the second one is a

consistency server [3]. First the block clutches entire file data and metadata. Then, without elucidating metadata, it dispenses a trouble-free interface for user to store variable-sized data blocks and later recoup them by cryptographic hash [3]. Without manufacturing signed messages justifiable under a user's public key, malicious aggressor cannot interfere with a user's information, paradoxically even if given complete database server to unfaithful provider. In spite of this guarantee, SUNDR executes within a fair-minded element of existing timid network File systems.

The Fork Join Causal is a modest failing of causal consistency that can be both risk-free and live nevertheless defective nodes. Then in a second layer of Depot two layer architecture, a Depot contrivances protocol that utilizes this unchanging ordering of updates to dispense other appealing consistency, hardness, permanence, and amelioration properties. The cost of these assurances is unpretentious and that Depot can permit defects and preserve excellent attainability, latency, aloft and hardness even when not worthy faults transpires [4]. SPORC, SUNDR, DEP are reasonable systems but impediment of this systems is these system does not underpin implementation of SQL statements over encrypted data.

B. Distributing Data Among Distant Providers

V. Ganapathy, D. Thomas, T. Feder, H. Garcia-Molina, and R. Motwani proposed system such as distributing data among different providers. Precursory system SPORC [2], SUNDR [3], DEP [4] was established using abstract ideas of data encryption which generates a massive system aloft. To conquer this stumbling block distributing data for privacy is novel proposal designs.

This novel scheme scatters data among non-associated cloud service providers to attain data seclusion. To scatter data among non-associated cloud service providers heuristic greedy hill climbing algorithm is utilized. Segregating the query at the user to queries for the diverse sites is done by beginning of a hierarchy or process upwards algorithms are furnished [5]. Ultimately at the sites are united to acquire solved. But cloud database service providers have the opportunity or possibility to reattain all information by directly collaborating with each other. This is pitfall of this system.

C. Transparent Data Encryption Feature

Dr. Anwar Pasha Deshmukh, Riyazuddin Qureshi proposed the system Transparent Data Encryption feature. Transparent Data Encryption of the database file is performed at the page level. Encrypted database are encrypted before they are written to disk and decrypted when read into memory [9].

First upon we need to create a master key, and then obtain a certificate protected by the master key after that create a database encryption key and protect it by the certificate and Set the database to use encryption. The figure shows the steps of Transparent Data Encryption [9]. A DBMS engine proffers the practicability of encrypting data at the file

system level. However, the DBMS is trusted and decrypts data before their use. Hence, this modus operandi is not pertinent to the database-as-a-service environment because we presume that the cloud provider is unfaithful storage [9].

D. CryptDB Architecture

R.A. Popa, C.M.S. Redfield, N. Zeldovich, and H. Balakrishnan proposed the system Proxy-based architecture. CryptDB stratagem is to effectuate queries atop of encrypted data and the key enlightenment that makes it pragmatic is that SQL utilizes a clearly Defined set of operators; those are having the power to support productively over encrypted data [10].

CryptDB constructed as proxy-based architecture which permit the implementation of SQL operations over encrypted data through SQL aware encryption algorithms. This implementation is identical to the SQL operation over unencrypted data. This architecture is reliance on middle level proxy. This middle level allows a user to communicate to the cloud Database-as-a-service to orchestrate it for reading from, writing to cloud database. Nevertheless, it promotes divers weaknesses. The reason that the proxy is faithful middle level, its purposes cannot be contracted out to a cloud database vendor. Consequently, the proxy is meant to be effectuated and supervised by the cloud user. But in many occasions middle level proxy can become a solitary of disaster and a system obstruction [11].

In a proxy-based architecture mandatory that any user activity should drift through one middle level server is not satisfactory to cloud-based plot, in which multiple users, quintessentially dispensed among different areas, entail simultaneous access to data stored in the same cloud database

III. ARCHITECHTURE DESIGN

Protected Database-as-a-Service design such a way that multiple users can connect to cloud database through protected Database client dashboard or API . Users are logged in using their cloud service ID and password provided by cloud vendor then client logged in into Protected Database-as-a-Service using special ID and password provided by protected database as a service which would be used as a "Master Key".

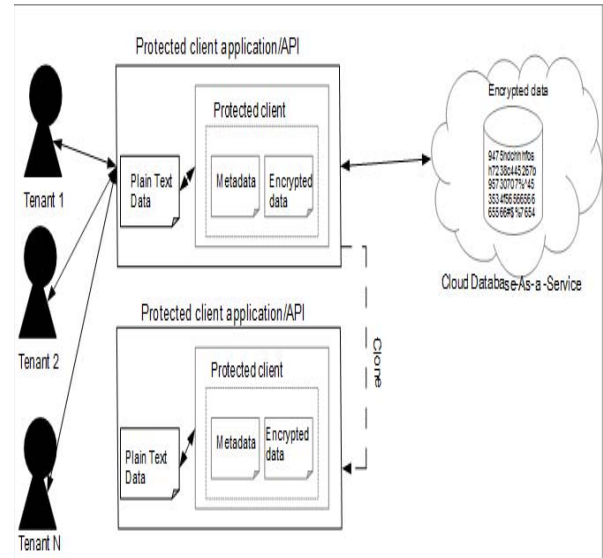


Figure 1. Protected Database-as-a-Service Architecture [11]

Protected Database-as-a-Service allows every user to connect to the cloud database, to read from the cloud database, write to the cloud database and administer the cloud database. Protected Database-as-a-Service architecture is divided into two parts: 1) Cloud database: Protected Database-as-a-Service uses various encryption policies to encrypt data. Encrypted data and related metadata is stored on the cloud database. A database metadata is also get encrypted with the plain text data, Plain text data and encrypted data are stored together on the cloud database. Unlike previously proposed system where database metadata is stored at the client side. 2) Protected Database-as-a-Service Client: Protected Database-as-a-Service client maintains the information which includes metadata, encrypted data and plaintext data. A plain text data is data which user wants to process and store on the cloud database. Metadata and encrypted data gives information about data which is stored on the cloud. It helps user to write to and read from cloud database.

IV. METHODOLOGY

A. Encryption Strategy

In this the users plain text data is stored in relational database table. To conserve information user data is encrypted by applying varied types of encryption algorithms. Then encrypted user data stored into cloud database through protected table [11]. Each plain text table then encrypted to execute intelligible SQL statement on encrypted database table which is stored on the cloud database. Plain text table and table name encrypted using same encryption algorithm and key encryption policy. Protected table holds field protected column. In Protected Database-as-a-service whole database, fields and table name get encrypted for more security, in

some previously proposed system only information stored in database is encrypted.

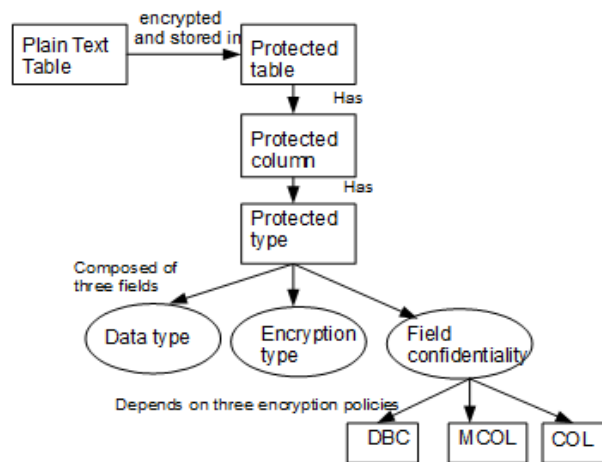


Figure 2. Proposed Encryption Strategies used in Protected Database-as-a-Service

Cloud database attributes remain unencrypted. From plain text table attributes it is uncomplicated to guess secret information stored in cloud database. Protected Database-as-a-Service not only encrypts information stored into database table but also it encrypts the attribute of cloud database. Generally columns are encrypted using randomly generated key. Protected column again holds protected type field which is composed of three fields:

- 1) Data type: The data type field gives information about type of column plain text.
- 2) Encryption type: This field represents type of encryption algorithm is used to encrypt respected column field.

Encryption type is as follows:

- a) Plain: This encryption type permits to store unencrypted data in the cloud database and allows operations of any type of SQL operator.
 - b) OPE: It dispense Order Preserving Encryption that allows the implementation of range queries and disproportion operators. Immense part of our application implementation is based on the OPE algorithm.
 - c) DET: This encryption type encrypts data through algorithms that assurance similar relations from the plaintext state to the encryption state. Consequently, it braces the accomplishment of equality checks and aggregation operators
 - d) Random: Random (RND) encryption type pledges the extortionate data secrecy. Random encryption type is effectuated through the standard AES algorithm in CBC mode using a random initialization value [11].
- 3) Field confidentiality: Field confidentiality field gives information about key used to encrypt the column field. To avoid complication or instead of using random key for each column when columns shares some similar information same key is used to encrypt such a column. These keys are decided as per the three predefined confidentiality attributes give below:

- a) COL: Random key
- b) Multi Col (MCOL): Same key for column with the join operation or foreign key constrain.
- c) DB: Special key for multiple columns are involve in same operation.[11]

B. Metadata Management Strategy

The information obligatory to manage SQL statements is managed by Protected Database-as-a-Service. Whole database metadata is store inside the metadata management table. Metadata management table divides in two type of metadata table those metadata tables maintain by Protected Database-as-a-Service. Figure 3, shows the conceptual diagram of metadata management strategy.

There are three metadata tables are maintained by Protected Database-as-a-Service:

- 1) Database Metadata Table: This metadata table not accommodates any information related to user database. It acts as a key ring. Database metadata table holds information about key encryption policies and data types.
- 2) Table Metadata: Table metadata is associated with each protected table. It holds metadata related to protected table such as name of secure table and unencrypted name of protected table. Also holds information about encryption key used to encrypt the database, database type and field confidentiality policy. Table metadata also contains column metadata which include some sub fields:

- a) Plain name: This field holds column name of plain text table.
- b) Coded name: Encrypted name of column.
- c) Secure type: Type of encryption policy associated with column.
- d) Encryption key: Key used for encryption.

C. Metadata Storage Table

Metadata storage table stores and encrypt table metadata and database metadata using same key. This encryption key is introduced as “Master key”. Authenticated users who knows master key can encrypt and decrypt the data. Each master key accessories with ID which further becomes primary key of metadata storage table. Message Authentication Code (MAC) function used to compute user ID. User can retrieves metadata of table using its plain text name.

V. CONCLUSIONS

Proposed Protected Database-as-a-Service approach, gives solution which promises confidentiality of indescribable information. Unlike avant-garde approaches propound system does not on single middle level proxy which could be tern system to be disaster. Its attainable to access geographically disperses encrypted information from the cloud data using SQL commands. This system architecture does not required abatement to the cloud database. It can be promptly apply on the existing system. There are no conceptual and empirical constraint to enlarged this system or append contemporary algorithms in

this system. Read and write function with no structure moderation demand insignificant aloft.

ACKNOWLEDGMENT

We would like to thank Prof.Vivek N. Waghmare, Information Technology Department, Sandip Institute of Technology and Research Centre, Nasik for his persistent guidance.

REFERENCES

- [1] H.Hacigu "mu"s.,B.Iyer,andS.Mehrotra,"ProvidingDatabaseas a Service," Proc. 18th IEEE Int'l Conf. Data Eng., Feb. 2002..
- [2] A.J. Feldman, W.P. Zeller, M.J. Freedman, and E.W. Felten, "SPORC: Group Collaboration Using Untrusted Cloud Resources," Proc. Ninth USENIX Conf. Operating Systems Design and Implementation, Oct. 2010.
- [3] J. Li, M. Krohn, D. Mazie`res, and D. Shasha, "Secure Untrusted Data Repository (SUNDR)," Proc. Sixth USENIX Conf. Opearting Systems Design and Implementation, Oct. 2004.
- [4] P. Mahajan, S. Setty, S. Lee, A. Clement, L. Alvisi, M. Dahlin, and M. Walfish, "Depot: Cloud Storage with Minimal Trust," ACM Trans. Computer Systems, vol. 29, no. 4, article 12, 2011.
- [5] V. Ganapathy, D. Thomas, T. Feder, H. Garcia-Molina, and R. Motwani, "Distributing Data for Secure Database Services," Proc. Fourth ACM Int'l Workshop Privacy and Anonymity in the Information Soc., Mar. 2011.
- [6] G. Aggarwal, M. Bawa, P. Ganesan, H. Garcia-Molina, K. Kenthapadi, R. Motwani, U. Srivastava, D. Thomas, Y. Xu" Two Can Keep a Secret: A Distributed Architecture for Secure Database Services" Proceedings of the 2005 CIDR Conference.
- [7] Curino, Carlo et al. "Relational Cloud: A Database-as-a-Service for the Cloud." 5th Biennial Conference on Innovative Data Systems Research, CIDR 2011, January 9-12, 2011 Asilomar, California.
- [8] H. Hacigu "mu"s., B.Iyer, C.Li, andS.Mehrotra, "Executing SQL over Encrypted Data in the Database-Service-Provider Model," Proc. ACM SIGMOD Int'l Conf. Management Data, June 2002.
- [9] Dr. Anwar Pasha Deshmukh, Riyazuddin Qureshi" Transparent Data Encryption- Security of Database Using Microsoft SQL Server 2008 and Oracle", IJCSt Vol. 2, ISSue 2, June 2011
- [10] R.A. Popa, C.M.S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting Confidentiality with Encrypted Query Processing," Proc. 23rd ACM Symp. Operating Systems Principles, Oct. 2011.
- [11] Luca Ferretti, Michele Colajanni, and Mirco Marchetti,"Distributed, Concurrent, andIndependent Access to Encrypted Cloud Databases", IEEE Transaction on Parallel and Disdistributed Systems, VOL. 25, No. 2, Feb 2014.